

# Parallel RCOUNT Verification with Serial Equivalence

Gio Della-Libera

Draft — June 28, 2026

## Abstract

Public RCOUNT packages should be checkable outside election offices, including when full-election packages are large enough that verification work should be parallelized. This paper documents the first parallel verifier slice in `rcount-core`. The claim is not a benchmark result. It is an equivalence contract: for the implemented parallelized check family, `verify_package_parallel` must produce the same `VerificationReport` as `verify_package`. The current surface parallelizes contest selection-sum checks with Rayon while preserving the serial order of `EquationPass` records, then appends the remaining verifier checks in the same order as the serial entry point. The fixtures cover both accepted packages and a failed selection-sum package, so the parallel path is constrained to keep transcript semantics rather than merely to return a matching boolean.

## 1 Introduction

RCOUNT packages are meant to be public evidence, not office-only artifacts. Anyone who receives the package should be able to run the verifier and recover the same transcript that an election office would see. For full-election packages, that goal creates an ordinary engineering pressure: independent checks over many public records should eventually run in parallel.

Parallel verification is only useful if it does not change what is being verified. A faster verifier that reorders reports, hides a failing equation, or returns a coarser pass/fail result would weaken the audit surface. The public object is the transcript: the ordered list of equations that passed, plus the same error behavior when a package fails.

This paper documents the first RCOUNT parallel verifier slice. The serial entry point is `verify_package`; the parallel entry point is `verify_package_parallel`. The implementation uses Rayon through `rayon::prelude` and parallelizes the contest selection-sum family first. The research claim is deliberately narrow: the parallel verifier preserves the serial transcript semantics for the implemented surface. It does not yet claim measured speedups.

## 2 Equivalence Contract

The contract is serial equivalence, not approximate agreement. For a package in the current implemented surface, the two verifier entry points must satisfy:

| Property               | Meaning                                                                                                                            |
|------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| Same pass records      | The <code>passed</code> vector contains the same <code>EquationPass</code> records.                                                |
| Same order             | Those records appear in the same order that <code>verify_package</code> would have emitted.                                        |
| Same failure behavior  | A package rejected by the parallelized check family is rejected with the same <code>RcountCoreError</code> as the serial verifier. |
| Same remaining surface | Checks not yet parallelized are appended by the parallel entry point in the same sequence as the serial entry point.               |

In code, an `EquationPass` contains an equation id, contest id, and reporting-unit id. Transcript equality therefore means more than both verifiers returning success. It means the public report names the same equations for the same contests and reporting units in the same deterministic sequence.

The selection-sum checks may execute concurrently, but their reports are collected into a vector before the rest of the verifier surface is appended. That vector is the boundary where parallel work is converted back into the serial transcript order. The remaining checks continue to run through the same serial functions, so the first parallel slice is constrained to be a drop-in replacement for the opening selection-sum loop.

### 3 Implemented Surface

The serial verifier, `verify_package`, first validates contests, then walks `package.summaries`. For each summary it looks up the contest, runs `verify_contest_selection_sum`, and pushes an `EquationPass` with equation id `contest_selection_sum`. After that opening loop, it extends the report with status-event, batch, lineage, reference, privacy, CVR-summary, audit, RLA, jurisdiction-adaptor, and manual-audit checks.

The parallel verifier, `verify_package_parallel`, keeps the same contest validation and the same later extension sequence. Only the summary loop is replaced. It calls `par_iter()` over `package.summaries`, performs the same contest lookup and selection-sum equation check, collects the result as `Result<Vec<EquationPass>, RcountCoreError>`, and then extends the report with that vector before running the remaining serial checks.

| Check                                | Failure caught or preserved                                                                                                |
|--------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| <code>contest_selection_sum</code>   | First parallelized family; edited summary selection totals fail in both entry points with the same error.                  |
| <code>EquationPass</code> ordering   | Parallel summary checks are collected before the later verifier families are appended, preserving the public report order. |
| <code>verify_package</code>          | Baseline serial transcript and error behavior.                                                                             |
| <code>verify_package_parallel</code> | Parallel entry point that must equal the serial <code>VerificationReport</code> on accepted fixtures.                      |

This is intentionally the smallest useful parallelization boundary. It exposes Rayon in `rcount-core` through `rayon::prelude`, but it does not move source-hash, batch-accounting, or audit checks into parallel execution. Those checks remain part of the equivalence report

because the parallel entry point still runs them in serial after the collected selection-sum passes.

## 4 Fixtures

The positive equivalence test is `parallel_verifier_matches_serial_reports`. It builds a list of synthetic packages that exercise the broader verifier surface: `synthetic_summary_basic_package`, `synthetic_canvass_correction_package`, `synthetic_mail_batch_added_package`, `synthetic_precinct`, `synthetic_privacy_inclusion_package`, `synthetic_cvr_summary_package`, `synthetic_rla_replay_package`, `synthetic_rla_stopping_package`, `synthetic_rla_margin_package`, `synthetic_rla_statistical_package`, `synthetic_colorado_rla_package`, `synthetic_california_rla_package`, and `synthetic_manual_audit`. For each package, it runs `verify_package` and `verify_package_parallel`, then asserts `assert_eq!(parallel, serial)`.

That assertion is the important fixture shape. It compares the whole `VerificationReport`, not just the number of passed checks. Since the report derives equality over its `passed` vector and `failed` vector, the fixture catches ordering drift as well as missing or extra `EquationPass` records.

The failure equivalence test is `parallel_verifier_matches_serial_error_for_bad_selection_sum`. It uses `synthetic_bad_selection_sum_package` and compares `verify_package_parallel(&package).expect_err(...)` with `verify_package(&package).expect_err(...)`. This pins the negative case to the same error value for the first parallelized check family.

Together, the two tests make the first parallel surface a transcript-equivalent implementation detail: accepted fixtures must produce identical reports, and a known bad selection sum must fail as the serial verifier fails.

## 5 Boundaries

This paper is an equivalence and determinism note. It is not a performance benchmark. The plan for this slice explicitly leaves package-size and timing benchmarks as future work, so this paper reports no timings, no speedups, and no package-size thresholds. Performance is the motivation for the interface, not an empirical result claimed here.

The implementation boundary is also narrow. The first parallelized family is `contest_selection_sum`. Source-hash checks, batch and accounting checks, audit-algorithm replay, RLA checks, jurisdiction adapters, and manual audit checks still run through the same serial functions after the collected selection-sum pass vector. Parallelizing those families is future work only where their transcript order can remain deterministic.

The contract does not change RCOUNT's evidence model. It does not make a package more correct than the serial verifier would. It only says that the parallel entry point is allowed to exist when it preserves the exact public transcript semantics of the serial entry point for the implemented check family.

## 6 Conclusion

RCOUNT now has a first parallel verifier slice with an explicit serial equivalence contract. `verify_package_parallel` parallelizes the selection-sum loop with Rayon, collects the resulting `EquationPass` records, and then appends the rest of the verifier surface in the same order as `verify_package`. The fixture suite compares complete reports for accepted packages and compares the serial and parallel errors for a bad selection-sum package.

That is the right first claim for public election evidence. Before measuring throughput, the verifier must prove that parallel execution is not a semantic change. The remaining work is to add real package-size and timing benchmarks, then extend parallelism to source-hash, batch, and accounting families only where the same deterministic transcript rule can be maintained.

## References

- Gio Della-Libera. Rcount overview: Reproducible election count packages. Technical report, Apportionment Research, 2026a.
- Gio Della-Libera. Parallel rcount verification with serial equivalence. Technical report, Apportionment Research, 2026b.